Product Spaces

Michael Betancourt

April 2023

All probability theory chapters can be found here.

A repository containing all of the files used to generate this chapter is available on GitHub.

Table of contents

Sometimes a system of applied interest can be adequately modeled with a single space. When a system is too complicated for any one single space, however, we may be able to model it with *multiple spaces* at the same time. Product spaces integrate multiple spaces together by first combining their underlying sets and then their associated structures.

1 Product Sets

Product sets combine elements from multiple sets into *composite* elements. To develop this concept as cleanly as possible, we'll first investigate how to combine elements from finite sets before considering the general case.

1.1 Finite Product Sets

Consider two finite sets, one with three elements,

$$X_1 = \{\square, \clubsuit, \diamondsuit\},$$

and one with two elements,

$$X_2 = \{\heartsuit, \spadesuit\}.$$

One way to combine these two sets together is to collect their individual elements together into larger set,

$$X_1 \cup X_2 = \{\square, \clubsuit, \diamondsuit, \heartsuit, \spadesuit\}.$$

This *concatenated* set allows us to choose from any of the elements in X_1 and X_2, but we can choose *only one element at a time*.

In order to choose elements from *both sets at the same time*, we need to account for all of the possible *pairs* of elements from X_1 and X_2. Placing the elements from X_1 before those from X_2 gives the pairs

$$\{(\square, \heartsuit), (\square, \spadesuit), (\clubsuit, \heartsuit), (\clubsuit, \spadesuit), (\diamondsuit, \heartsuit), (\diamondsuit, \spadesuit)\}.$$

This set of ordered pairs is referred to as the **Cartesian product** of X_1 and X_2, or a **product set** $X_1 \times X_2$ with **component sets** X_1 and X_2 (Figure 1a).

The order of the component sets matters for this construction. Reversing the order of the component sets results in the pairs (Figure 1b),

$$X_2 \times X_1 = \{(\heartsuit, \square), (\spadesuit, \square), (\heartsuit, \clubsuit), (\spadesuit, \clubsuit), (\heartsuit, \diamondsuit), (\spadesuit, \diamondsuit)\}.$$

Consequently, $X_1 \times X_2$ and $X_2 \times X_1$ are *distinct* product sets.

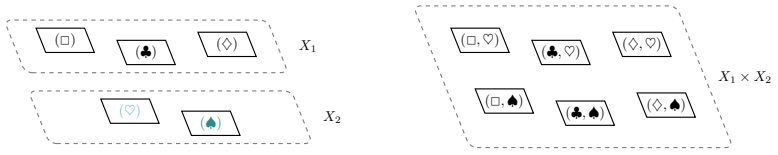
By definition, each element of a binary product set is uniquely specified by one element of X_1 and one element of X_2. Consequently, every variable x taking values in the product set $X_1 \times X_2$ can be represented by an ordered pair of component variables,

$$x = (x_1, x_2),$$

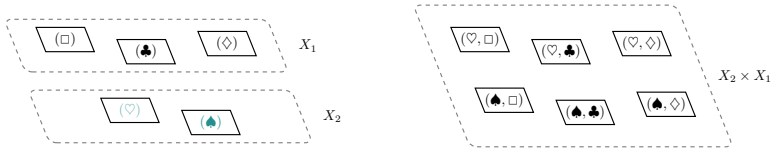
with $x_1 \in X_1$ and $x_2 \times X_2$.

Again, order is important here. A variable x taking values in the product set $X_2 \times X_1$ is instead comprised of the the component variables

$$x = (x_2, x_1).$$



(a)



(b)

Figure 1: (a) The product set $X_1 \times X_2$ consists of all ordered pairs combining one element from X_1 and one element from X_2. (b) Reversing the order of the pairs gives the product space $X_2 \times X_1$.

1.2 General Product Sets

This construction immediately generalizes to any finite number of component sets. Given I component sets indexed in a particular order

$$\{X_1, \ldots, X_i, \ldots X_I\},$$

we can construct a corresponding product set

$$X_1 \times \ldots \times X_i \times \ldots \times X_I = \prod_{i=1}^{I} X_i$$

from all of the ways that we can select one element from each component set in that specified order,

$$(x_1, \ldots, x_i, \ldots, x_I) \in \prod_{i=1}^{I} X_i$$

with $x_i \in X_i$. In other words, a variable taking values in a finite product set is comprised of a *sequence* of component variables.

In theory, we could combine the component sets in any order. In practice, however, it's easier to combine them in the order of their indices, relabeling the indices if we ever need a different ordering.

Because product elements are specified by multiple component elements, product sets are sometimes referred to as **multivariate** sets. The component sets themselves are referred to in a variety of different ways, including **degrees of freedom** and **coordinates**.

Conveniently, this construction generalizes immediately to a countably infinite number of component sets: every element of a countably-infinite product set is uniquely determined by an element from each of the countably-infinite component sets. Defining product sets from an uncountably infinite number of component sets requires just a little bit more care. To ensure that choosing an uncountably infinite number of elements at the same time is well-defined, we have to invoke the infamous axiom of choice.

1.3 Coordinating Indices

Many applications require working with multiple product variables at the same time. A common strategy for differentiating between these different variables is to label them with integer indices; for example we might write

$$x_1, x_2 \in \prod_{i=1}^{I} X_i.$$

These indices, however, are easy to confuse with the indices used to denote the component variables of which each product variable is comprised.

In applications where indexing distinct product variables is useful, I will use a comma to separate the two types of indices, with the first index always labeling the different variables and the latter always labeling the different components,

$$x_j = (x_{j,1}, \ldots, x_{j,i}, \ldots, x_{j,I}) \in \prod_{i=1}^{I} X_i.$$

In other words, $x_{j,i}$ refers to the ith component variable of the jth product variable.

1.4 Projection Functions

Conveniently, we can always recover the behavior of each the component elements from a given product element. Consider, for example, the component sets X_1 and X_2, the binary product space $X_1 \times X_2$, and the product variable (x_1, x_2).

Ignoring the second component variable x_2 leaves just x_1, which specifies a unique element of X_1. In other words, each element of $X_1 \times X_2$ defines a unique element of X_1. We can formalize this relationship as a function between the two sets,

$$\begin{aligned} \varpi_1 : X_1 \times X_2 &\rightarrow X_1 \\ (x_1, x_2) &\mapsto x_1 \, . \end{aligned}$$

At the same time, ignoring the first component variable leaves x_2, which specifies a unique element of X_2. This, in turn, defines a relationship between $X_1 \times X_2$ and X_1 which we can encode in *another* function,

$$\begin{aligned} \varpi_2 : X_1 \times X_2 &\to X_2 \\ (x_1, x_2) &\mapsto x_2 \, . \end{aligned}$$

More generally, ignoring all but one of the component variables in

$$(x_1, \ldots, x_I) \in X^{1:I}$$

defines a map from the product set to the corresponding component set,

$$\begin{aligned} \varpi_i : \prod_{i=1}^{I} X_i &\to X_i \\ (x_1, \ldots, x_i, \ldots, x_I) &\mapsto x_i \, . \end{aligned}$$

Because these I functions *project* the composite product set to an individual component set, they are known as **projection functions**. When the component sets are referred to as "coordinates", the projection functions are also referred to as **coordinate functions**.

Projection functions are useful for compactly organizing the composite structure of product sets. We will be returning to them often.

2 Thinned Product Sets

The product set $\prod_{i=1}^{I} X_i$ is not the only product set that we can construct from the component sets

$$\{X_1, \ldots, X_I\}.$$

Any selection of these component sets defines its own product set.

More formally, any sequence of $J \leq I$ distinct component indices,

$$\mathsf{i} = (i_1, \ldots, i_j, \ldots, i_J)$$

with

$$i_j < i_{j+1},$$

defines a product set

$$\prod_{i \in \mathsf{i}} X_i = \prod_{j=1}^{J} X_{i_j}.$$

Each element of this product set is uniquely specified by an element in each of the corresponding component sets,

$$(x_{i_1}, \ldots, x_{i_J}) \in \prod_{i \in \mathsf{i}} X_i$$

with

$$x_{i_j} \in X_{i_j}.$$

Because these product sets are comprised of only some of the available component sets, I will refer to them as **thinned product sets**.

There are many ways of selecting from the available component sets, and hence many thinned product sets that we can construct in addition to the full product set. In order to distinguishing between all of the possibilities, especially when we are working with enough component sets that explicit enumeration is unfeasible, we'll need to develop some careful notation.

2.1 Index Subsequences

Given an initial sequence, we can construct **subsequences** by removing some elements while maintaining the order of the remaining elements. Using this terminology, the component indices that define each thinned product set are subsequences of the sequence of all available component indices,

$$(1, \ldots, i, \ldots, I).$$

For example, from the initial sequence of component indices

$$(1, 2, 3)$$

we can construct six distinct, non-empty subsequences,

$$(1), (2), (3),$$
$$(1, 2), (1, 3), (2, 3).$$

The three atomic sequences identify one of the component sets, but each of the three binary sequences defines a thinned product set,

$$X_1 \times X_2, \quad X_1 \times X_3, \quad X_2 \times X_3.$$

I will denote the relationship between a sequence i and a subsequence i'

$$\mathsf{i}' \vec{\subset} \mathsf{i}.$$

Here $\subset$ communicates that the sequence on the left-hand side contains fewer elements than the sequence on the right-hand side, while the arrow communicates the consistent ordering.

Subsequences are basically subsets equipped with a persistent ordering. It should be no surprise, then, that we can generalize all of the usual subset operations to subsequences provided that we respect the ordering.

For example, we can define the intersection of two subsequences as the elements shared by both sequences in the appropriate order. Starting with the initial sequence

$$(1, 2, 3, 4),$$

the intersection of the two subsequences

$$(1, 2, 4)$$

and

$$(1, 3, 4)$$

is the subsequence

$$(1, 4).$$

I will denote this ordered intersection as

$$(1, 2, 4) \vec{\cap} (1, 3, 4) = (1, 4),$$

with the arrow once again communicating the consistent ordering of subsequences.

Similarly, we can define an ordered union of two subsequences as the appropriately ordered combination of their respective elements,

$$(1, 3) \vec{\cup} (3, 4) = (1, 3, 4).$$

These operations allow us to define a variety of useful manipulations of a given sequence, such as partitions. Given an initial sequence i, a **sequence partition** is a collection of K subsequences

$$\mathsf{i}_k \vec{\subset} \mathsf{i}$$

that are nonempty

$$\mathsf{i}_k \neq (),$$

mutually disjoint,

$$\mathsf{i}_k \vec{\cap} \mathsf{i}_{k'} = ()$$

for $k \neq k'$, and span the full sequence,

$$\vec{\cup}_{k=1}^{K} \mathsf{i}_k = \mathsf{i}.$$

2.2 Multi-Index Notation

Given a collection of component sets, we can construct the full product set as well as a myriad of thinned product sets. **Multi-index** notation neatly organizes all of these possibilities by labeling each product set with the corresponding sequence of component indices.

Specifically, we denote the thinned product set defined by the sequence of component indices

$$\mathsf{i} = (i_1, \ldots, i_J),$$

using a superscript,

$$X^{\mathsf{i}} \equiv \prod_{i \in \mathsf{i}} X_i.$$

Similarly, we denote a variable taking values in this particular thinned product set with a subscript,

$$x_{\mathsf{i}} \in X^{\mathsf{i}}.$$

The full product set built up from all available component indices is used so often that it deserves its own special notation. If we denote the sequence of all component indices by

$$1{:}I = (1, \ldots, I),$$

then we can denote the full product set as

$$X^{1:I} = \prod_{i \in 1:I} X_i = \prod_{i=1}^{I} X_i.$$

Similarly, we can denote the corresponding product variable as

$$x_{1:I} \in X^{1:I}.$$

Consider, for example, the component sets X_1, X_2, X_3, X_4, and X_5. Using multi-index notation we can denote the full product set as

$$X_1 \times X_2 \times X_3 \times X_4 \times X_5 = X^{1:5}$$

with product variables

$$x_{1:5} = (x_1, x_2, x_3, x_4, x_5).$$

At the same time, if we select only the component indices

$$\mathsf{i} = (1, 3, 4),$$

then we can construct the thinned product set

$$X^{\mathsf{i}} = X^{(1,3,4)} = X_1 \times X_3 \times X_4$$

with product variables

$$x_{\mathsf{i}} = x_{(1,3,4)} = (x_1, x_3, x_4).$$

Multi-index notation is especially useful when working with multiple product sets at the same time. For instance, let's return to our previous example and then introduce another subsequence of component indices,

$$\mathsf{i}' = (1, 2, 5).$$

We can now define three distinct product sets,

$$\begin{aligned} &X_1 \times X_2 \times X_3 \times X_4 \times X_5, \\ &X_1 \times X_3 \times X_4, \\ &X_1 \times X_2 \times X_5. \end{aligned}$$

Using multi-index notation, we can write out these product spaces more compactly as

$$X^{1:5}, \quad X^{(1,3,4)}, \quad X^{(1,2,5)}.$$

This becomes even more compact if we use the names of the ordered subsets of component indices,

$$X^{1:5}, \quad X^{\mathsf{i}}, \quad X^{\mathsf{i}'}.$$

Even with only the relatively small number of component spaces available here, multi-index notation results in much more compact equations.

The multi-index notation that I have defined here is a variant of the multi-index notation popular in physics. This notation, however, is not particularly common in probability theory or statistics. That said, I will be using is extensively in this book.

2.3 Generalized Projection Functions

By dropping the unused component elements, any element of the full product set defines an element of *any* thinned product set. For example, given the product element

$$(x_1, x_2, x_3) \in X^{(1,2,3)},$$

we can define not only the component elements

$$x_1 \in X_1, x_2 \in X_2, x_3 \in X_3,$$

but also the thinned product elements

$$(x_1, x_2) \in X^{(1,2)}, (x_1, x_3) \in X^{(1,3)}, (x_2, x_3) \in X^{(2,3)}.$$

The relationship between full product elements and thinned product elements defines generalized projection functions. Specifically, any subsequence of component indices

$$\mathsf{i} \vec{\subset} 1 : I,$$

defines a corresponding projection function

$$\varpi_{\mathsf{i}} : X^{1:I} \to X^{\mathsf{i}}.$$

We can even construct projection functions between compatible thinned product sets. If

$$\mathsf{i}' \vec{\subset} \mathsf{i}$$

then we can define the projection operators

$$\varpi^{\mathsf{i}}_{\mathsf{i}'} : X^{\mathsf{i}} \to X^{\mathsf{i}'}.$$

3 Product Subsets

Just as an element in each of the component sets defines a unique element from the corresponding product set, a subset in each of the component sets defines a unique subset of the product set. Not every subset of a product set, however, can be constructed in this way.

3.1 Rectangles

Specifying a subset in each of the component sets,

$$\mathsf{x}_i \subseteq X_i,$$

defines a unique subset of the product set,

$$\mathsf{x}_1 \times \cdots \times \mathsf{x}_i \times \cdots \times \mathsf{x}_I = \times_{i=1}^{I} \mathsf{x}_i \subset X^{1:I}.$$

The particular subsets of a product set of this form are known as **product subsets**. We are, unfortunately, responsible for not confusing these product subsets with arbitrary subsets of a product set.

Many of the important subsets of a product set can be constructed in this way. The empty set of a product set, for example, is given by combining the component empty sets,

$$\emptyset = \times_{i=1}^{I} \emptyset_i.$$

Similarly, the full product set is given by combining the full component sets,

$$X^{1:I} = \times_{i=1}^{I} X_i.$$

Not every subset of $X^{1:I}$, however, is a product subset. More formally, the product power set is *not* the product of the component power sets,

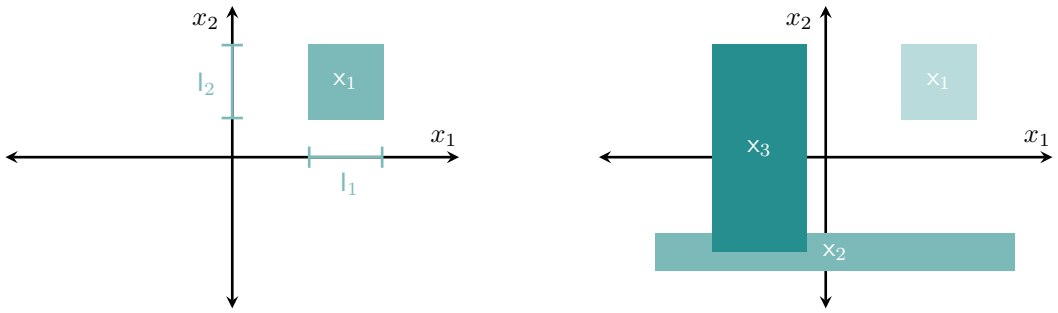
$$2^{\prod_{i=1}^{I} X_i} \neq \prod_{i=1}^{I} 2^{X_i}.$$

In fact, the product power set is bigger than the product of the component power sets,

$$2^{\prod_{i=1}^{I} X_i} \supset \prod_{i=1}^{I} 2^{X_i}.$$

Typically the product power set is *much* bigger than the product of the component power set; in this case, most of subsets of the product set are not product subsets!

On the product set $\mathbb{R} \times \mathbb{R}$, the product of finite intervals defines geometric rectangles (Figure 2). In analogy, more general product subsets are often referred to as **rectangle subsets**, even when the geometric intuition doesn't quite carry over. Despite the geometric exaggeration, using "rectangle" instead of "product subset" is quite useful for avoiding confusion between product subsets and more general subsets of a product set.



(a) (b)

Figure 2: (a) On the product set $\mathbb{R} \times \mathbb{R}$, geometric rectangles are defined by the product two component interval subsets I_1 and I_2. (b) Different component interval subsets result in a diversity of rectangular shapes.

3.2 Cylinders

Rectangle, however, is by no means the only geometric analogy that we can abuse. Consider the product set $S \times \mathbb{R}$. Here, the product of a circle S and a linear interval $I \subset \mathbb{R}$ defines a geometric cylinder (Figure 3a).

Analogizing this geometry motivates a class of subsets on *any* product set. A **generalized cylinder subset**, or just **cylinder subset** for short, is defined to be a rectangle subset where all but a *finite number* of component subsets equals the full component set (Figure 3b).

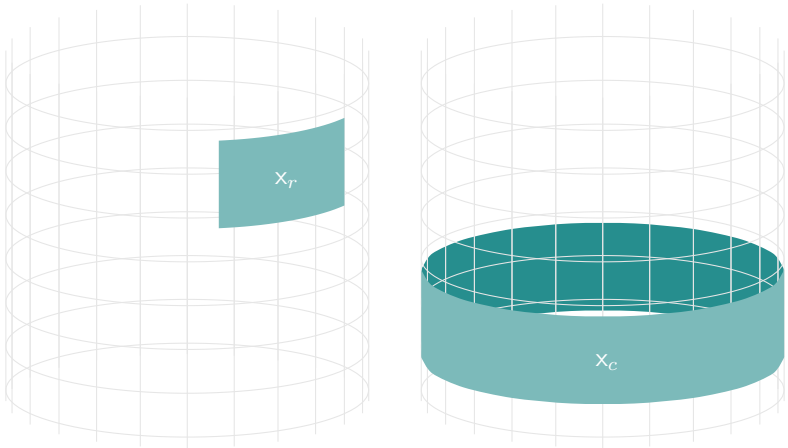
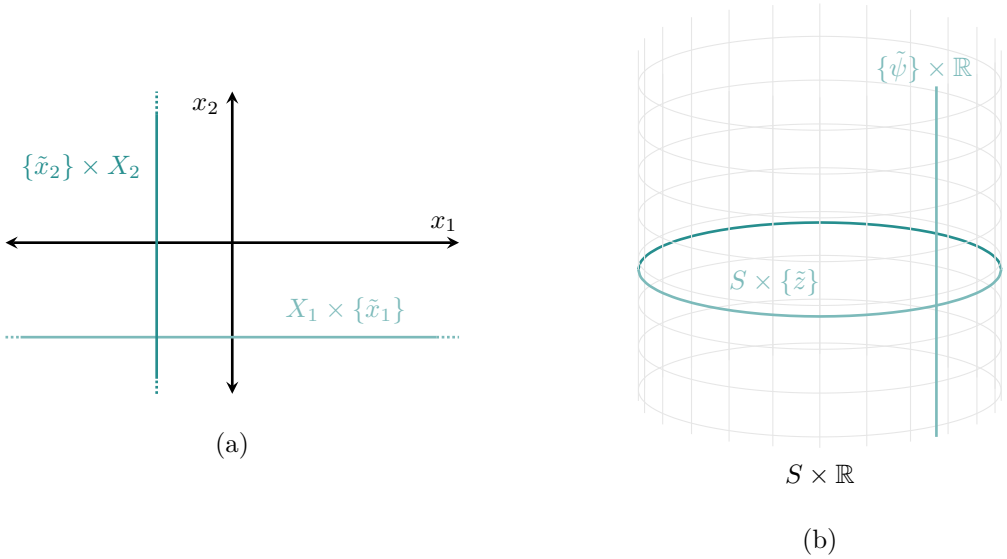


Figure 3: (a) On the product set $S \times \mathbb{R}$, geometric rectangles, such as x_r, are defined by products of angular intervals $\mathsf{I}_2 \subset S$ and linear intervals $\mathsf{I}_1 \subset \mathbb{R}$. (b) Geometric cylinders, such as x_c, are a special case where the angular interval covers all of S. The term "cylinder subset" is used more generally for any product subset where all but a finite number of component subsets equal the full component set.

Note that, by this definition, there is no strict difference between rectangle subsets and cylinder subsets on a product set with only a finite number of component sets. The distinction, however, becomes critical when considering product sets built up from an infinite number of component sets. Here, cylinder subsets ensure that only a finite number of components are actively contributing to the behavior of the product subset at any given time.

3.3 Cross Sections

When the component subsets are either full or atomic, with at least one component subset atomic, the resulting cylinder subset on the product space is known as a **cross section**. Cross sections model the behavior of a product set when we bind some, but not all, of the component sets to particular elements.



(a) (b)

Figure 4: Cross sections are a special type of cylinder subset where each component subset is either full or atomic, and at least one is atomic. (a) On the product set $\mathbb{R} \times \mathbb{R}$, cross sections trace out vertical and horizontal lines, and (b) on the product set $S \times \mathbb{R}$ cross sections trace out circles and vertical lines.

For instance, binding $\tilde{x}_1 \in X_1$ doesn't specify a point in the cross section $X_1 \times X_2$. It does, however, restrict the possibilities to the cross section

$$\{\tilde{x}_1\} \times X_2 \subset X_1 \times X_2.$$

Similarly, fixing $\tilde{x}_2 \in X_2$ restricts the product set to the cross section

$$X_1 \times \{\tilde{x}_2\} \subset X_1 \times X_2.$$

More generally, binding the component elements

$$\{\tilde{x}_i \mid i \in \mathsf{i}\}$$

for

$$\mathsf{i} \vec{\subset} 1 : I,$$

but leaving the complementary component elements

$$\{x_i \mid i \in \mathsf{i}^c\}$$

free, defines a cross section of the product set $X^{1:I}$.

The most awkward aspect of cross sections is compactly denoting which components have been constrained. I will often write cross sections symbolically as

$$X^{\mathsf{i}^c} \times \{x_{\mathsf{i}}\} \subset X^{1:I},$$

with the understanding that the component objects have to be reordered as necessary. If, for example, $I = 5$ and $\mathsf{i} = (2, 4)$, then

$$\begin{aligned} X^{\mathsf{i}^c} \times \{x_{\mathsf{i}}\} &= X^{(1,3,5)} \times \{x_{(2,4)}\} \\ &= X_1 \times \{x_2\} \times X_3 \times \{x_4\} \times X_5. \end{aligned}$$

Cross sections can also be written in terms of projection functions. Specifically, each level set of the projection function

$$\varpi_{\mathsf{i}} : X^{1:I} \to X^{\mathsf{i}}$$

defines a unique cross section,

$$\varpi_{\mathsf{i}}^{-1}(\tilde{x}_{\mathsf{i}}) = X^{\mathsf{i}^c} \times \{\tilde{x}_{\mathsf{i}}\}.$$

Once we bind the component elements

$$\{\tilde{x}_i \mid i \in \mathsf{i}\},$$

an element of the full product set is uniquely determined by any configuration of the remaining component elements

$$\{x_i \mid i \in \mathsf{i}^c\}.$$

Consequently, these complementary component elements specify elements of the cross section

$$X^{\mathsf{i}^c} \times \{\tilde{x}_{\mathsf{i}}\} \subset X^{1:I}.$$

These elements, however, also define an element of the thinned product set X^{i^c}.

In other words, every element of a cross section is uniquely identified with an element of the corresponding thinned product set. More formally, we have a natural bijection between the two,

$$X^{\mathsf{i}^c} \times \{x_{\mathsf{i}}\} \cong X^{\mathsf{i}^c}.$$

More intuitively, we can always interpret the cross sections $X^{\mathsf{i}^c} \times \{x_{\mathsf{i}}\}$ as distinct copies of X^{i^c}, each of which is distinguished by an element of the thinned product set

$$x_{\mathsf{i}} \in X^{\mathsf{i}}.$$

The cross sections

$$X_1 \times X_2 \times \{x_3\} \subset X_1 \times X_2 \times X_3,$$

for instance, each behave like a copy of the thinned product set

$$X_1 \times X_2.$$

Once we've fixed $x_3 \in X_3$, any element of the resulting cross section defines a corresponding element of the thinned product and vice versa. This allows us to jump between a more global perspective – cross sections as subsets of the full product set – and a more local perspective – thinned product sets – at our convenience.

While each cross section can be identified with a common thinned product set, the *behavior* within each cross section at any given time can, in general, be different. Consequentially, we have to be careful to respect the individuality of each cross section.

3.4 Rectangular Subset Operations

Most of the subset operations are not compatible with the composite structure of a rectangle subset.

The complement of a rectangle subset, for example, is not generally given by applying the complement operation to each of the component subsets (Figure 5),

$$\mathsf{x}^c \neq \times_{i=1}^{I} \mathsf{x}_i^c.$$

Instead the product of the x_i^c typically includes fewer elements than the complement of the product of the x_i^c,

$$\times_{i=1}^{I} \mathsf{x}_i^c \subseteq \left(\times_{i=1}^{I} \mathsf{x}_i \right)^c.$$

Similarly, the union of two rectangle subsets,

$$\mathsf{x} = \times_{i=1}^{I} \mathsf{x}_i$$

and

$$\mathsf{x}' = \times_{i=1}^{I} \mathsf{x}_i',$$

is not generally given by the product of the component unions (Figure 6),

$$\mathsf{x} \cup \mathsf{x}' \neq \times_{i=1}^{I} \mathsf{x}_i \cup \mathsf{x}_i'.$$

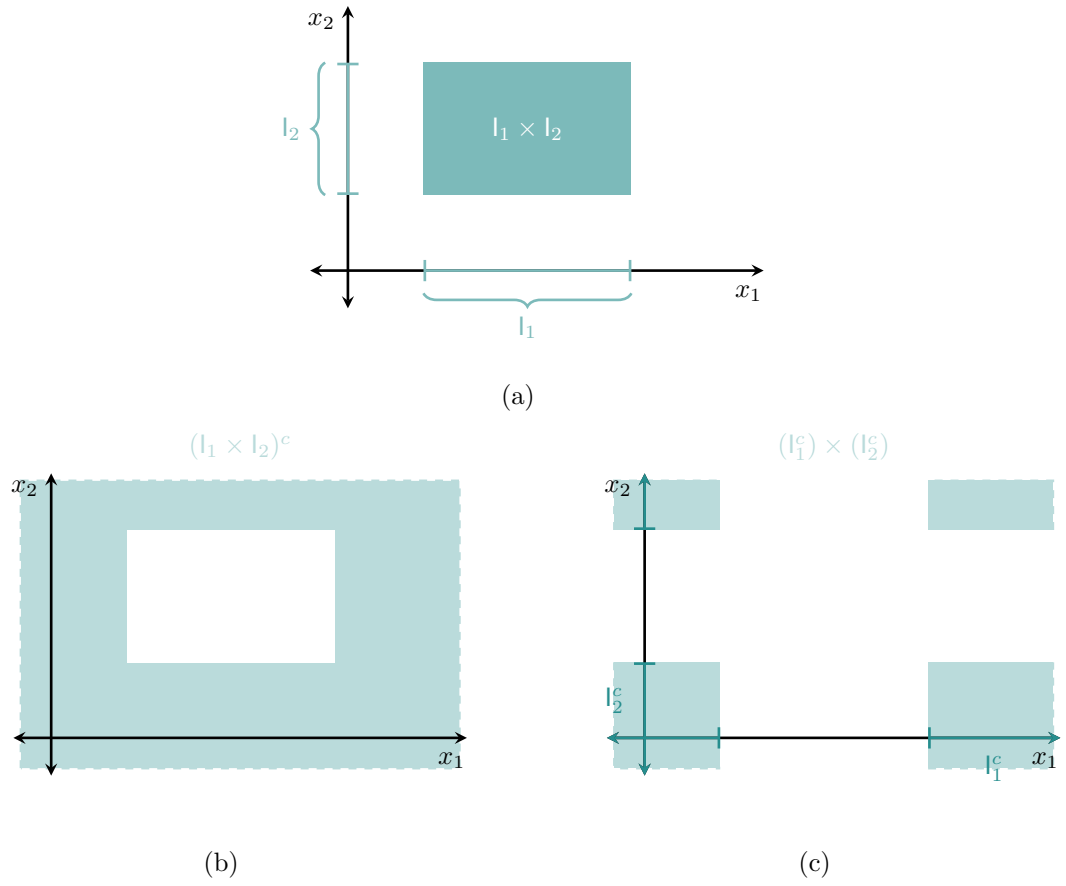


Figure 5: (a) The construction of a rectangle subset from component subsets doesn't commute with the complement operator. (b) Taking the product first and then applying the complement operator typically doesn't result in another rectangular subset (c). Applying the complement operator to each component subset and then taking their product, however, does.

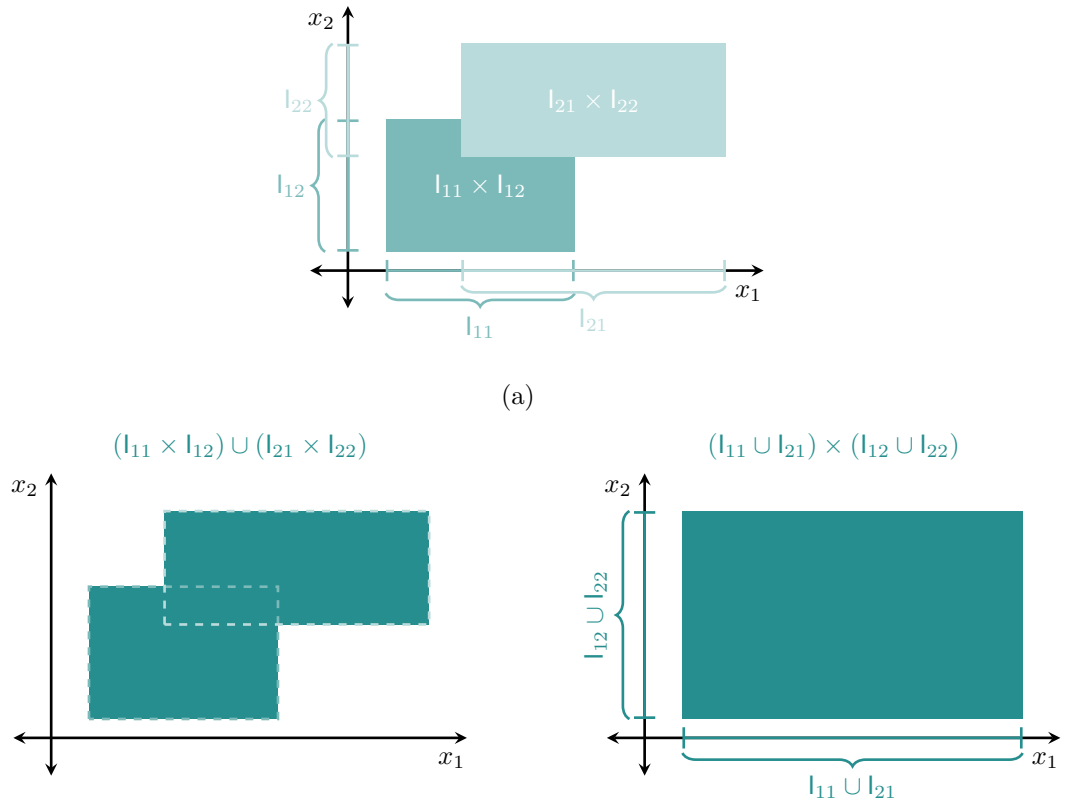


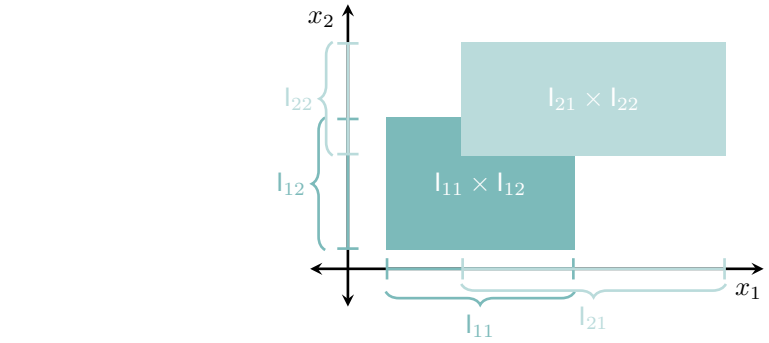
Figure 6: The union operator also doesn't commute with (a) the construction of rectangle subsets. (b) Taking the product first and then applying the union operator usually results in a non-rectangle subset that is smaller (c) than the rectangular subset given by applying the union operator to each component subset and then taking their product.

In general, the product of the component unions is not a rectangle subset. Moreover, it it larger than the union of the products,

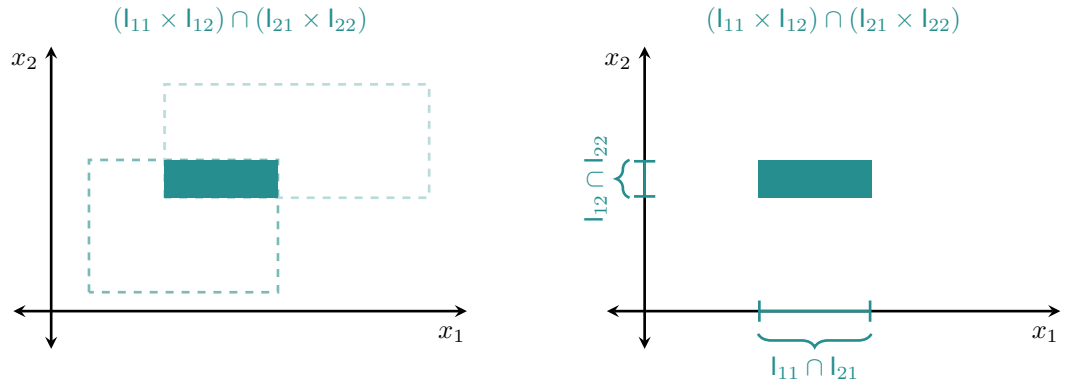
$$\mathsf{x} \cup \mathsf{x}' \subseteq \times_{i=1}^{I} \mathsf{x}_i \cup \mathsf{x}_{i'}.$$

The lone exception is the intersection of rectangle subsets, which can always be derived from the intersection of the component subsets (Figure 7),

$$\mathsf{x} \cap \mathsf{x}' = \times_{i=1}^{I} \mathsf{x}_i \cap \mathsf{x}'_i.$$



(a)



(b) (c)

Figure 7: Of the three subset operators, only the intersection operator commutes with (a) the construction of rectangle subsets. (b) Taking the product first and then applying the intersection operator always gives the same rectangle subset as (c) applying the intersection operator to each component subset and then taking their product.

An immediate consequence of these properties is that rectangle subsets are closed under intersections but not unions and complements. While closure is often useful in practice, in this case

the lack of closure is actually quite useful. In particular, it allows us to build up non-rectangle subsets by applying set operations to rectangle subsets. We'll come back to this point in Section 5.4.

4 Decomposing Product Sets

The composite structure of product sets allows us to slice and dice them into smaller pieces. Because the smaller pieces are often easier to directly manipulate, these decompositions can be extremely useful in practical applications.

4.1 Two Component Sets

Let's start as simply as possibly by considering the two component sets X_1 and X_2 and their product $X_1 \times X_2$.

4.1.1 Decomposition

Decomposing X_1 into the union of its atomic subsets,

$$X_1 = \bigcup_{x_1 \in X_1} x_i,$$

induces a similar decomposition of the product set into a union of cross sections, (Figure 8a),

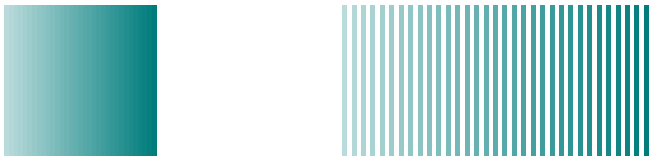
$$\begin{aligned} X_1 \times X_2 &= \left(\bigcup_{x_1 \in X_1} \{x_i\} \right) \times X_2 \\ &= \bigcup_{x_1 \in X_1} \left(\{x_1\} \times X_2 \right). \end{aligned}$$

Because these cross sections can be interpreted as copies of X_2, we can interpret this as a decomposition of the product set into repeated instances that component set.

This procedure can also be applied to the second component set, resulting in the decomposition (Figure 8b)

$$X_1 \times X_2 = \bigcup_{x_2 \in X_2} \left(X_1 \times \{x_2\} \right).$$

Binary product spaces can be sliced in either direction!



$X_1 \times X_2$ $\{\{x_1\} \times X_2 \text{ for } x_1 \in X_1\}$

(a)

$X_1 \times X_2$ $\{X_1 \times \{x_2\} \text{ for } x_2 \in X_2\}$

(b)

Figure 8: (a) When we decompose X_1 into atomic subsets, the product set $X_1 \times X_2$ decomposes into a union of cross sections, $\{x_1\} \times X_2$, each of which behaves like a replication of the component set X_1. (b) The product set can also be sliced in the other direction, resulting in the cross sections $X_1 \times \{x_2\}$.

4.1.2 Composition

Cross sections can also be used to construct product sets in the first place. Consider, for instance, starting with only the component set X_1. Introducing a copy of X_2 for each $x_1 \in X_1$ defines the cross sections $\{x_1\} \times X_2$, and the union of these cross sections is exactly the product set $X_1 \times X_2$. Equivalently, if we start with X_2 then we can construct $X_1 \times X_2$ by introducing a copy of X_1 for each $x_2 \in X_2$.

Indeed, this construction is basically how ordered pairs are formally defined in the first place. In abstract set theory, an ordered pair is defined by an element from one of the component sets and a compatible, unordered pair of component elements,

$$(x_1, x_2) \equiv \{x_1, \{x_1, x_2\}\}$$

or

$$(x_1, x_2) \equiv \{x_2, \{x_1, x_2\}\}.$$

Given the initial component element, the compatible component pairings effectively defines a cross section.

4.1.3 Sequential Specification

These procedures offers some geometric insight into how we *sequentially* specify elements of a product set. Selecting a component element

$$\tilde{x}_1 \in X_1$$

doesn't fully determine an element from the product set. It does, however, restrict the possibilities to the cross section

$$\{\tilde{x}_1\} \times X_2 \subset X_1 \times X_2.$$

To completely determine a product element, we need to select product element from the corresponding cross section (Figure 9a)

$$\tilde{x} \in \{\tilde{x}_1\} \times X_2.$$

Alternatively, we can select an element from the second component set,

$$\tilde{x}_2 \in X_2,$$

and then an element from the cross section (Figure 9b),

$$\tilde{x} \in X_1 \times \{\tilde{x}_2\}.$$

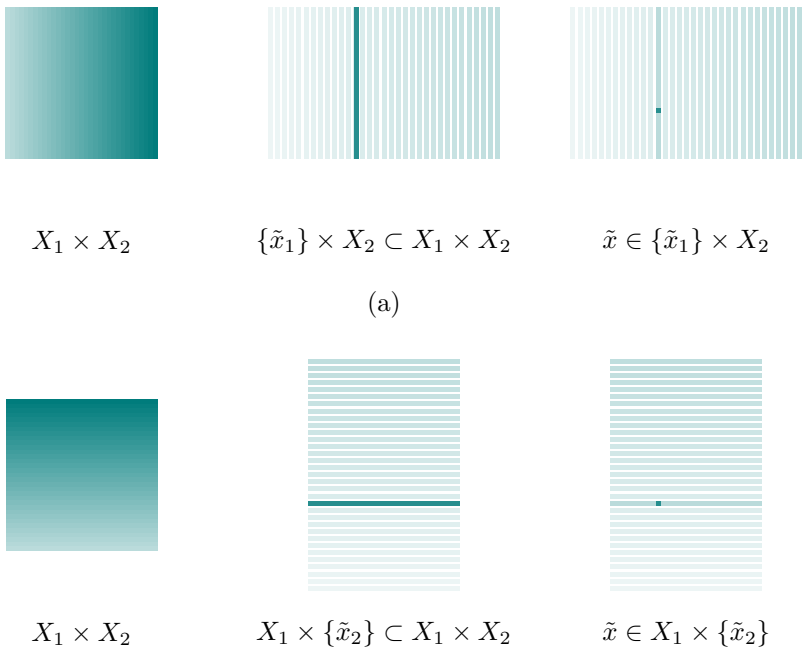


Figure 9: A product set element $(\tilde{x}_1, \tilde{x}_2) \in X_1 \times X_2$ can be sequentially constructed in two different ways. (a) Selecting $\tilde{x}_1 \in X_1$ restricts the possible elements to the cross section $\{\tilde{x}_1\} \times X_2$. (b) Alternatively, we can first select $\tilde{x}_2 \in X_2$ before selecting a point from the corresponding cross section, $\tilde{x}_1 \in X_1 \times \{\tilde{x}_2\}$. Either way, we don't have to consider the product set in its entirely.

4.2 Three Component Sets

Increasing the number of component sets dramatically increases the ways that we can slice and dice the resulting product set. Before jumping in more generally, let's see what happens when we consider just one more component set to bring us to X_1, X_2, and X_3.

4.2.1 Decomposition Into Binary Product Sets

Expanding the first component set into its atomic subsets induces the cross section decomposition

$$X_1 \times X_2 \times X_3 = \bigcup_{x_1 \in X_1} \{x_1\} \times X_2 \times X_3.$$

In this case, the product set decomposes into replications of the binary product set $X_2 \times X_3$.

At the same time, expanding the other two component sets gives the alternative decompositions (Figure 10)

$$X_1 \times X_2 \times X_3 = \bigcup_{x_1 \in X_1} \{x_1\} \times X_2 \times X_3$$

and

$$X_1 \times X_2 \in X_3 = \bigcup_{x_3 \in X_3} X_1 \times X_2 \times \{x_3\}.$$

4.2.2 Decomposition Into Component Sets

We can also expanding binary products of component sets into atomic subsets. This yields three more decompositions of the full product set (Figure 11),

$$\begin{aligned}
X_1 \times X_2 \times X_3 &= \bigcup_{(x_1, x_2) \in X_1 \times X_2} \{x_1\} \times \{x_2\} \times X_3 \\
X_1 \times X_2 \times X_3 &= \bigcup_{(x_1, x_3) \in X_1 \times X_3} \{x_1\} \times X_2 \times \{x_3\} \\
X_1 \times X_2 \times X_3 &= \bigcup_{(x_2, x_3) \in X_2 \times X_3} X_1 \times \{x_2\} \times \{x_3\}.
\end{aligned}$$

We can slice three-component product sets into not only binary product sets but also individual component sets!

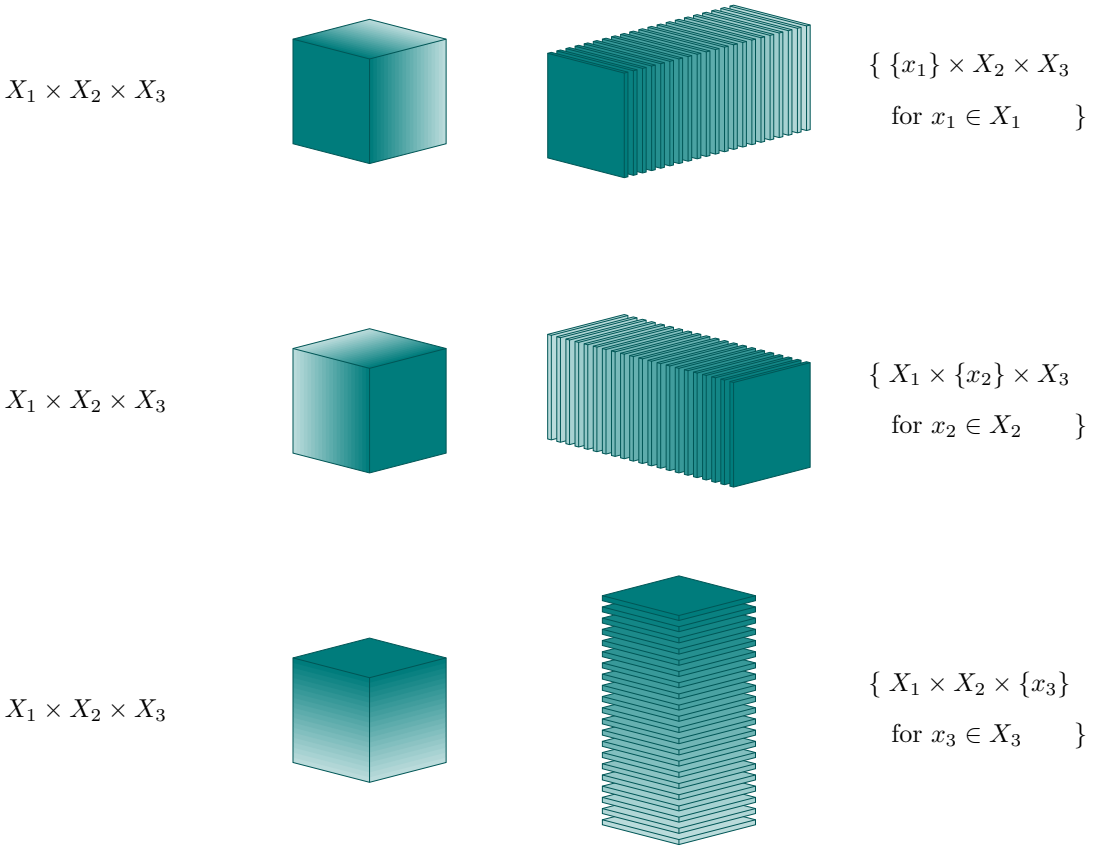


Figure 10: Isolating any one of its component sets decomposes the product set $X_1 \times X_2 \times X_3$ into replications of binary product sets. Slicing along X_1, for example, decomposes the product set into the cross sections $\{x_1\} \times X_2 \times X_3$, each of which can be treated as a distinct copy of $X_2 \times X_3$.

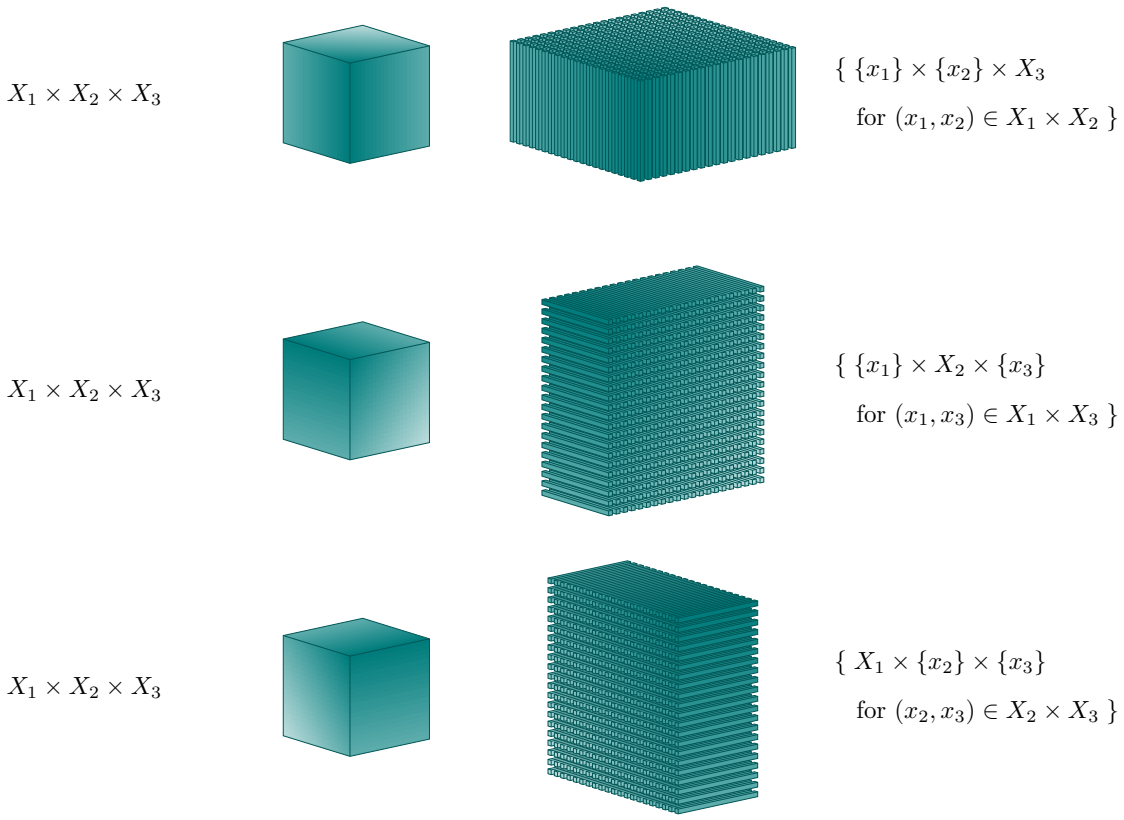


Figure 11: Isolating two of its component sets at the same time decomposes the product set $X_1 \times X_2 \times X_3$ into replications of the single, remaining component set. Jointly slicing along X_1 and X_2, for example, dices the product set into the cross sections $\{x_1\} \times \{x_2\} \times X_3$, each of which can be treated as a distinct copy of X_3.

4.2.3 Recursive Decomposition

Some of these decompositions can be further *refined*. Consider, for instance, the decomposition

$$X_1 \times X_2 \times X_3 = \bigcup_{(x_1,x_2)\in X_1\times X_2} \{x_1\} \times \{x_2\} \times X_3.$$

The set that indexes each copy of X_3 can be further decomposed into

$$X_1 \times X_2 = \bigcup_{x_1\in X_1} \{x_1\} \times X_2.$$

This is equivalent to decomposing the full product set into

$$\bigcup_{x_1\in X_1} \bigcup_{x_2\in\{x_1\}\times X_2} \{x_1\} \times \{x_2\} \times X_3.$$

Equivalently, we can decompose $X_1 \times X_2$ into

$$X_1 \times X_2 = \bigcup_{x_2\in X_2} X_1 \times \{x_2\},$$

which decomposes the full product set into

$$\bigcup_{x_2\in X_2} \bigcup_{x_1\in X_1\times\{x_2\}} \{x_1\} \times \{x_2\} \times X_3.$$

4.2.4 Composition

Each of these decompositions motivates a different way to sequential construct a product set.

For example, we can lift X_1 to $X_1 \times X_2 \times X_3$ by introducing a copy of $X_2 \times X_3$ for each element $x_1 \times X_1$. Alternatively, we can start with X_2 and introduce copies of $X_1 \times X_3$ or start with X_3 and introduce copies of $X_1 \times X_2$.

At the same time, if we start with $X_1 \times X_2$ then we can construct $X_1 \times X_2 \times X_3$ by introducing a copy of the third component set X_3 for each ordered pair $(x_2, x_3) \in X_1 \times X_2$. Starting with $X_1 \times X_3$ or $X_1 \times X_2$ requires replicating X_2 or X_1, respectively.

Finally, we can build up the full product set from smaller pieces using more steps. We could, for instance, start with X_1 and then introduce a copy of X_2 for each initial element to define $X_1 \times X_2$. At this point, we can replicate X_3 for each ordered pair $(x_2, x_3) \in X_1 \times X_2$. Each permutation of the component indices defines a unique construction path.

4.2.5 Sequentially Specifying Product Elements

Once again, these decompositions offer geometric insight into what is happening as we sequentially specify elements of the full product set. For example, we might specify an element of the first component set,

$$\tilde{x}_1 \in X_1$$

and then an element of the cross section

$$\tilde{x} \in \{x_1\} \times X_2 \times X_3.$$

In practice, this is equivalent to selecting x_1 and then x_2 and x_3 at the same time.

Alternatively, we might start by specifying the pair

$$(\tilde{x}_1, \tilde{x}_2) \in X_1 \times X_2$$

before selecting from the cross section

$$\tilde{x} \in \{x_1\} \times \{x_2\} \times X_3.$$

This is equivalent to selecting x_1 and x_2 at the same time and then selecting x_3.

We might also specify component elements one at a time (Figure 12), starting with

$$\tilde{x}_1 \in X_1,$$

then

$$\tilde{x}_2 \in \{\tilde{x}_1\} \times X_2,$$

and finally

$$\tilde{x} \in \{x_1\} \times \{x_2\} \times X_3 \subset \{\tilde{x}_1\} \times X_2$$

In practice, this corresponds to selecting x_1 before x_2 and then lastly x_3.

Across the last three examples, I have maintained the same order of the component sets. More generally, however, we can consider the component sets in any order. For example,

$$\begin{aligned} \tilde{x}_2 &\in X_2 \\ \tilde{x} &\in X_1 \times \{x_2\} \times X_3, \end{aligned}$$

$$\begin{aligned} (\tilde{x}_2, \tilde{x}_3) &\in X_2 \times X_3 \\ \tilde{x} &\in X_1 \times \{x_2\} \times \{x_3\}, \end{aligned}$$

and

$$\begin{aligned} \tilde{x}_2 &\in X_2 \\ \tilde{x}_3 &\in \{\tilde{x}_2\} \times X_3 \\ \tilde{x} &\in X_1 \times \{x_2\} \times \{x_3\} \end{aligned}$$

are all valid, but by no means exhaustive, ways to build up an element of a product set.

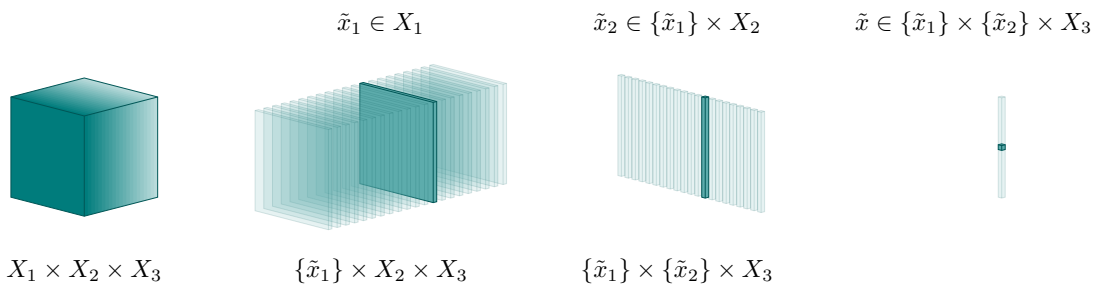


Figure 12: Specifying component elements one by one builds up an element of the product set sequentially. For instance, taking $\tilde{x}_1 \in X_1$ restricts the possible elements to the cross section $\{\tilde{x}_1\} \times X_2 \times X_3$. At this point, selecting an element $\tilde{x}_2 \in X_2$ restricts this cross section to the more refined cross section $\{x_1\} \times \{x_2\} \times X_3$. Finally, taking $\tilde{x}_3 \in X_3$ selects a product set element from this cross section.

4.3 General Case

Once we go beyond a few component sets, the number of ways that we can decompose of the resulting product set is so large that the possibilities can be overwhelming. Fortunately, multi-index notation makes it straightforward to specify any particular decomposition.

4.3.1 Direct Decomposition and Composition

Given a subsequence of component indices,

$$\mathsf{i} \vec{\subset} 1 : I,$$

we can define three product sets: the full product set, $X^{1:I}$, and two thinned product sets, X^{i} and X^{i^c}.

Decomposing the thinned product set X^{i} into its atomic elements,

$$X^{\mathsf{i}} = \bigcup_{x_{\mathsf{i}} \in X^{\mathsf{i}}} \{x_{\mathsf{i}}\},$$

decomposes the full product set into the corresponding cross sections,

$$X^{1:I} = \bigcup_{x_{\mathsf{i}} \in X^{\mathsf{i}}} X_{\mathsf{i}^c} \times \{x_{\mathsf{i}}\}.$$

At the same time, replicating X^{i^c} for each element of X^{i} lifts X^{i} into the full product set $X^{1:I}$. Heuristically, we can write this composition as

$$x = (x_{\mathsf{i}}, x_{\mathsf{i}^c}),$$

with the understanding that the component elements must be reordered as necessary.

For example, if $I = 5$ and $\mathsf{i} = (2, 4)$, then we would first select an element

$$x_{(2,4)} \in X^{(2,4)}$$

before selecting an element of

$$x_{(1,3,5)} \in X^{(1,3,5)}.$$

We could then write the resulting product set element as

$$\begin{aligned} x &= (x_{(2,4)}, x_{(1,3,5)}) \\ &= ((x_2, x_4), (x_1, x_3, x_5)) \\ &= (x_1, x_2, x_3, x_4, x_5). \end{aligned}$$

4.3.2 Recursive Decomposition and Composition

Decomposing $X^{1:I}$ into the cross sections

$$\bigcup_{x_{\mathsf{i}} \in X^{\mathsf{i}}} X_{\mathsf{i}^c} \times \{x_{\mathsf{i}}\}$$

can be impractical if the thinned product space X^{i} is still too overwhelming. Fortunately, nothing is stopping us from decomposing this space int more manageable pieces.

In particular, the subsequence of component indices

$$\mathsf{i}' \vec{\subset} \mathsf{i}$$

decomposes X^{i} into cross sections,

$$X^{\mathsf{i}} = \bigcup_{x_{\mathsf{i}'} \in X^{\mathsf{i}'}} X^{\mathsf{i}\,\mathsf{i}'} \times \{x_{\mathsf{i}'}\}.$$

If, at this point, the thinner product $X^{\mathsf{i}'}$ is still too much to handle, then we can iterate until we've broken the system down into sufficiently manageable pieces.

There are many ways that this recursive decomposition could proceed. Conveniently, each possibility is completely determined by a partition of the component indices $1 : I$ into disjoint subsequences

$$(\mathsf{i}_1, \ldots, \mathsf{i}_K)$$

with

$$\mathsf{i}_k \vec{\subset} 1 : I.$$

First, we iteratively remove the component indices in each i_k to define a sequence of thinning product sets, X^{j_k} with

$$\mathsf{j}_k = \vec{\cup}_{k'=k}^{K} \mathsf{i}_{k'}.$$

Next, we decompose the full product set into copies of X^{i_1},

$$X^{1:I} = \bigcup_{x_{\mathsf{j}_2} \in X^{\mathsf{j}_2}} X^{\mathsf{i}_1} \times \{x_{\mathsf{j}_2}\}.$$

At this point we can iterate $K-2$ more times, decomposing X^{j_k} into copies of X^{i_k},

$$X^{\mathsf{j}_k} = \bigcup_{x_{\mathsf{j}_{k+1}} \in X^{\mathsf{j}_{k+1}}} X^{\mathsf{i}_k} \times \{x_{\mathsf{j}_{k+1}}\}.$$

Each iteration peels off the component sets whose indices are in i_k.

Working through this subsequence partition backwards also allows us to construct the full product set in $K-1$ total steps. Starting with $k=K$, each iteration lifts the thinned product set X^{j_k} into the slightly less-thinned product set $X^{\mathsf{j}_{k-1}}$ by replicating $X^{\mathsf{i}_{k-1}}$.

For a concrete demonstration, consider $I=7$ and the sequence partition

$$(\mathsf{i}_1 = (2,4), \mathsf{i}_2 = (5), \mathsf{i}_3 = (1,6,7), \mathsf{i}_4 = (3)).$$

This gives

$$\begin{aligned}
\mathsf{j}_1 &= \mathsf{i}_1 \vec{\cup} \mathsf{i}_2 \vec{\cup} \mathsf{i}_3 \vec{\cup} \mathsf{i}_4 = (1,2,3,4,5,6,7) \\
\mathsf{j}_2 &= \mathsf{i}_2 \vec{\cup} \mathsf{i}_3 \vec{\cup} \mathsf{i}_4 = (1,3,5,6,7) \\
\mathsf{j}_3 &= \mathsf{i}_3 \vec{\cup} \mathsf{i}_4 = (1,3,6,7) \\
\mathsf{j}_4 &= \mathsf{i}_4 = (3).
\end{aligned}$$

With four subsequences, this partition defines three stages of decomposition. The first stage decomposes

$$\begin{aligned}
X^{\mathsf{j}_1} &= \bigcup_{x_{\mathsf{j}_{k+1}} \in X^{\mathsf{j}_{k+1}}} X^{\mathsf{i}_k} \times \{x_{\mathsf{j}_{k+1}}\} \\
X^{1:7} &= \bigcup_{x_{(1,3,5,6,7)} \in X^{(1,3,5,6,7)}} X^{(2,4)} \times \{x_{(1,3,5,6,7)}\}.
\end{aligned}$$

Then, at the second stage, we decompose

$$\begin{aligned}
X^{\mathsf{j}_2} &= \bigcup_{x_{\mathsf{j}_3} \in X^{\mathsf{j}_3}} X^{\mathsf{i}_2} \times \{x_{\mathsf{j}_3}\} \\
X^{(1,3,5,6,7)} &= X^{(5)} \times \{x_{(1,3,6,7)}\}.
\end{aligned}$$

Lastly, we decompose

$$X^{\mathsf{j}_3} = \bigcup_{x_{\mathsf{j}_4} \in X^{\mathsf{j}_4}} X^{\mathsf{i}_3} \times \{x_{\mathsf{j}_4}\}$$
$$X^{(1,3,5,6,7)} = X^{(1,6,7)} \times \{x_3\}.$$

At the same time, starting with

$$X^{\mathsf{j}_4} = X^3$$

and replicating

$$X^{\mathsf{i}_3} = X^{(1,6,7)}$$

defines

$$X^{\mathsf{j}_3} = X^{(1,3,6,7)}.$$

From here, replicating

$$X^{\mathsf{i}_2} = X^5$$

defines

$$X^{\mathsf{j}_2} = X^{(1,3,5,6,7)}.$$

Finally, replicating

$$X^{\mathsf{i}_1} = X^{(2,4)}$$

defines

$$X^{\mathsf{j}_1} = X^{(1,2,3,4,5,6,7)} = X^{1:7}.$$

5 Product Structure

In order to elevate product *sets* to product *spaces*, we need to equip products with structure. Conveniently, product structure can often derived from component structures. Moreover, this constructions is often reasonably straightforward.

In this section I will always assume the ambient product set $X^{1:I} = \prod_{i=1}^{I} X_i$ with component sets X_i.

5.1 Product Orderings

If each component set is equipped with a strict ordering, then we can unambiguously define a product variable $x \in X^{1:I}$ to be smaller than another product variable $x' \in X^{1:I}$ if *all* of the component elements in x are smaller than *all* of the corresponding component elements in x' (Figure 13a). In other words, $x < x'$ if and only if

$$x_i < x'_i$$

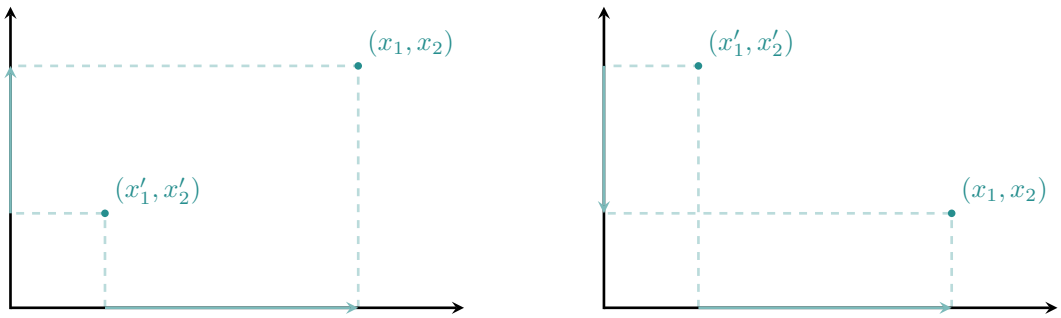


Figure 13: Strict orderings on each component set define only a partial ordering over product sets. (a) The element $x = (x_1, x_2)$ is unambiguously larger than the element $x' = (x'_1, x'_2)$ because both $x_1 > x'_1$ and $x_2 > x'_2$. (b) When the component orderings disagree, here $x_1 > x'_1$ but $x_2 < x'_2$, we cannot order the two elements and instead have to treat them as equivalent with respect to ordering.

for all $i \in 1, \ldots, I$.

On the other hand, if only *some* of the component elements in x are smaller than the corresponding component elements in x', with the other larger, then the comparison between the two product elements will be ambiguous (Figure 13b). Consequently, a strict ordering on the component sets does not fully define a strict ordering on the product set.

That said, we can use strict component orderings to define a *partial* ordering of the product set, where any two product elements with mixed component orderings are defined to be equivalent. In the same way, we can use partial component orderings to define a partial ordering of the product set. Either way, we are generally limited to partial product orderings.

5.2 Product Algebras

When each component set is equipped with an individual algebraic operation, we can define a corresponding product operation by applying these component-wise operations at the same time. For instance, if each component set is equipped with a binary operation

$$\begin{aligned} \cdot_i : X_i \times X_i &\to X_i \\ x_i, x'_i \quad &\mapsto x_i \cdot_i x'_i, \end{aligned}$$

then we can construct a **product operation** $\cdot : X^{1:I} \times X^{1:I} \to X^{1:I}$ as

$$x \cdot x' = (x_1 \cdot_1 x'_1, \ldots, x_i \cdot_i x'_i, \ldots, x_I \cdot_I x'_I).$$

In general, product operations acquire any properties that are shared by *all* of the component operations. For example, if all of the component operations are commutative, then the product operation will also be commutative. Similarly, if all of the component operations are unital with identity elements $x_{\mathrm{Id},i}$, then the product operation will also be unital with the composite identity element

$$x_{\mathrm{Id}} = (x_{\mathrm{Id},1}, \ldots, x_{\mathrm{Id},i}, \ldots, x_{\mathrm{Id},I}) \in X^{1:I}.$$

5.3 Product Metrics

Product metrics can be constructed by summing over the outputs of component metrics. More formally, if each component set is equipped with a metric

$$\begin{aligned} d_i : X_i \times X_i &\to \mathbb{R}^+ \\ x_i, x_i' &\mapsto d(x_i, x_i') \end{aligned}$$

then we can define a **product metric** as

$$\begin{aligned} d : X^{1:I} \times X^{1:I} &\to \mathbb{R}^+ \\ x, x' &\mapsto \sum_{i=1}^{I} d(x_i, x_i'). \end{aligned}$$

This construction ensures that the product metric satisfies all of the necessary properties of a metric. For example, the product distance vanishes if and only if all of the individual component distances vanish. This requires all of the component elements to be equal, which implies that the product elements are also equal. In other words, the product metric returns zero if and only if the two input product elements are the same, as required for a metric.

On the other hand, two input product elements are distinct if and only if at least one of the component element is distinct. In this case, at least one of the component distances will be greater than zero. Consequently, the summed component distances will be non-zero whenever the input product elements are distinct.

Symmetry of the product metric follows the commutativity of the component metrics,

$$\begin{aligned} d(x, x') &= \sum_{i=1}^{I} d(x_i, x_i') \\ &= \sum_{i=1}^{I} d(x_i', x_i) \\ &= d(x', x). \end{aligned}$$

Likewise, for any three product elements $x, x', x'' \in X$ we always have a triangle inequality,

$$\begin{aligned} d(x, x'') &= \sum_{i=1}^{I} d(x_i, x_i'') \\ &\leq \sum_{i=1}^{I} d(x_i, x_i') + d(x_i', x_i'') \\ &\leq \sum_{i=1}^{I} d(x_i, x_i') + \sum_{i=1}^{I} d(x_i', x_i'') \\ &\leq d(x, x') + d(x', x''). \end{aligned}$$

5.4 Product Topologies

When every component set is equipped with a component topology t_i, we can define open component subsets $\mathsf{x}_i \in \mathsf{t}_i$. The product of any combination of these open component subsets,

$$\prod_{i=1}^{I} \mathsf{x}_i,$$

defines candidates for open product subsets. In particular, these candidates include the product empty set and product full set as necessary for a topology.

By definition, any finite collection of open subsets in X_i,

$$\{\mathsf{x}_{1,i}, \ldots, \mathsf{x}_{j,i}, \ldots \mathsf{x}_{J,i}\} \in \mathsf{t}_i,$$

is closed under intersections,

$$\cap_{j=1}^{J} \mathsf{x}_{j,i} \in \mathsf{t}_i.$$

Consequently, these potentially-open product subsets will also closed under finite intersections,

$$\begin{aligned} \cap_j \mathsf{x}_j &= \cap_j \left(\times_{i=1}^{I} \mathsf{x}_{j,i} \right) \\ &= \times_{i=1}^{I} \left(\cap_j \mathsf{x}_{j,i} \right). \end{aligned}$$

Unfortunately, the union of any potentially-open product subsets will not, in general, be another potentially-open product subset. Combining the potentially-open product subsets with all of their unions, however, defines a collection of subsets that satisfies all of the properties of a topology. Because it is generated by rectangle subsets, this topology is known as the **box topology**.

When working with a finite number of component sets, the box topology inherits the nice features shared by all of the component topologies. Moreover, every projection function is continuous with respect to this topology.

These useful properties of the box topology, however, do not generalize to an infinite number of component sets. Conceptually, the box topology is too large to be productive once we move beyond a finite number of component sets.

Fortunately, we can construct a topology that remains well-behaved on *any* product set by refining the box topology. Instead of considering arbitrary products of open component subsets, we consider instead cylinder subsets built up from open component subsets. Including all of the finite intersections and arbitrary unions that we can generate from these open cylinder subsets defines what is conventionally referred to as a **product topology**.

If the number of component sets if finite, the box and product topologies are the same. When the number of component sets reaches infinity, however, the product topology becomes strictly smaller than the box topology. This smaller topology makes the product space more rigid, ensuring that for instance the projection functions remain continuous.

6 Transforming Product Spaces

We can always transform product spaces directly with monolithic maps that ignore any component structure. These functions map an entire input product set to an arbitrary output set,

$$\begin{aligned} f : X^{1:I} &\to Y \\ (x_1, \ldots, x_i, \ldots, x_I) &\mapsto y = f(x_1, \ldots, x_i, \ldots, x_I). \end{aligned}$$

For example, algebraic operations can be interpreted as functions from the input product set $X \times X$ to the output set X,

$$\begin{aligned} f : X \times X &\to X \\ (x_1, x_2) &\mapsto x = f(x_1, x_2). \end{aligned}$$

Metrics can also be interpreted as functions from that same input product set,

$$\begin{aligned} d : X \times X &\to [0, \infty] \\ (x_1, x_2) &\mapsto d(x_1, x_2). \end{aligned}$$

The classification of these functions, and the construction of pushforward and pullback functions, follows exactly the same as for functions with general input spaces. We can say much more, however, about functions that naturally harmonize with the component structure of an input product space.

6.1 Component Transformations

Transformations between two product spaces can always be decomposed into simpler transformations. Any function between two product spaces

$$\begin{aligned} f : X^{1:I} &\to Y^{1:J} \\ (x_1, \dots, x_i, \dots, x_I) &\mapsto (y_1, \dots, y_j, \dots, y_J) = f(x_1, \dots, x_i, \dots, x_I) \end{aligned}$$

is completely determined by J functions that map the input product set into each of the individual output component sets,

$$\begin{aligned} f_j : X^{1:I} &\to Y_j \\ (x_1, \dots, x_i, \dots, x_I) &\mapsto y_j = f_j(x_1, \dots, x_i, \dots, x_I). \end{aligned}$$

These component functions are given by composing f with each of the projection functions on the output space,

$$f_j = \varpi_j \circ f.$$

In general, these component functions mix the input components together to inform the behavior in each output component space. Some functions, however, allow each of the input component spaces to inform only a single output component space. More formally, if the input product space and output product space are built up from the same number of component spaces I, then some functions can be decomposed into component functions that map only one input component space into one output component space at a time,

$$\begin{aligned} f_i : X_i &\to Y_i \\ x_i &\mapsto y_i = f_i(x_i). \end{aligned}$$

In other words, these component-preserving functions transform each component *independently* of the others.

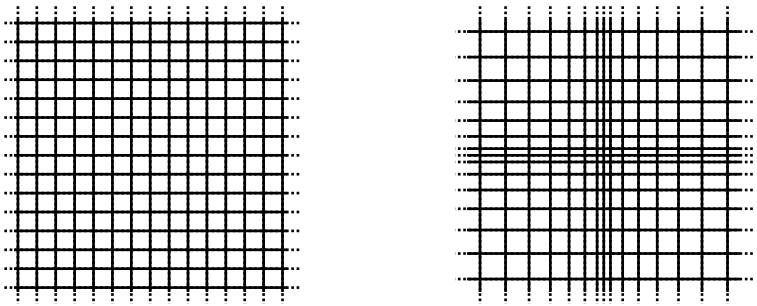
One nice feature of component-preserving transformations is that they preserve the orientation of grids defined from any component metric structure. For instance, the function

$$\begin{aligned} f : \mathbb{R} \times \mathbb{R} &\to \mathbb{R} \times \mathbb{R} \\ (x_1, x_2) &\mapsto (y_1, y_2) = (x_1^{\frac{3}{2}}, x_2^{\frac{3}{2}}) \end{aligned}$$

transforms x_1 into y_1 independently of the behavior of x_2, and transforms x_2 into y_2 independently of the behavior of x_1. While neither of these component transformations are isometries, the rectangular structure of the metric-informed grid is preserved (Figure 14a).

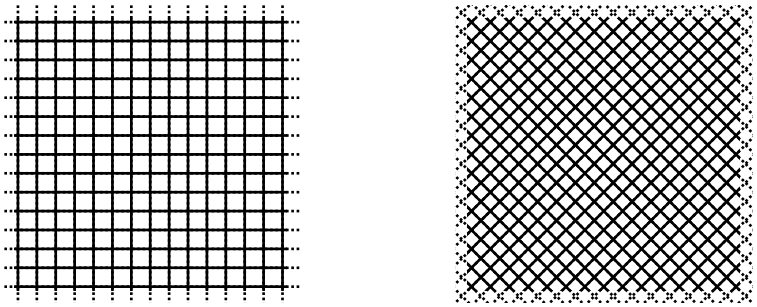
On the other hand, the function

$$\begin{aligned} f : \mathbb{R} \times \mathbb{R} &\to \mathbb{R} \times \mathbb{R} \\ (x_1, x_2) &\mapsto (y_1, y_2 = (x_1 + x_2, x_1 - x_2) \end{aligned}$$

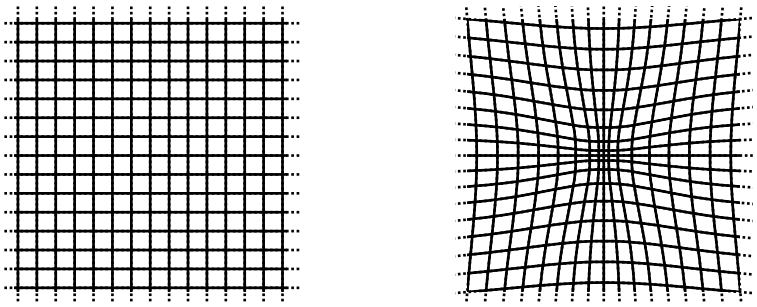


Input Grid Pushforward Grid

(a)



Input Grid Pushforward Grid



Input Grid Pushforward Grid

(b)

Figure 14: Maps between metric product spaces may or may not preserve the individual components, and this has consequences for how metric-informed grids transform. (a) Maps between metric product spaces that do preserve the product structure transform rectangular grids into rectangular grids. (b) On the other hand, maps that mix the components transform rectangular grids into grids that appear rotated, skewed, or otherwise warped.

mixes the input components x_1 and x_2 together, skewing the product structure in the process. Consequently, grids built up from the component metric structures on the input and output spaces will appear *warped* relative to each other (Figure 14b).

The behavior of component-preserving functions is completely determined by the behavior of the component functions. For example, a component-preserving function is injective, surjective, or bijective if and only if all of the component functions are injective, surjective, or bijective.

6.2 Partial Evaluation

Fully evaluating a function on a product space requires specifying a point in *every* component space. For instance, in order to evaluate the function

$$f : X_1 \times X_2 \to Y,$$

we need to specify both $\tilde{x}_1 \in X_1$ and $\tilde{x}_2 \in X_2$,

$$f(\tilde{x}_1, \tilde{x}_2) = y \in Y.$$

Providing only one component point leaves an empty slot in the input product space that we need to fill in order to complete the evaluation. For example, $f(\tilde{x}_1, x_2)$ is missing an a point in X_2 and $f(x_1, \tilde{x}_2)$ is missing a point in X_1. That empty slot, however, defines a relationship between the missing inputs and the output space.

Specifically, $f(\tilde{x}_1, x_2)$ implicitly defines a function that maps input points in the cross section $\{\tilde{x}_1\} \times X_2$ to output points in Y. Similarly, $f(x_1, \tilde{x}_2)$ implicitly defines a function that maps input points in the cross section $X_1 \times \{\tilde{x}_2\}$ to output points in Y.

More generally, consider the subsequence of component indices

$$\mathsf{i} = (i_1, \ldots, i_J)$$

and corresponding component elements

$$\tilde{x}_{\mathsf{i}} = (\tilde{x}_{i_1}, \ldots, \tilde{x}_{i_J}).$$

Inputting $\tilde{x}_{\mathsf{i}}$ into a function

$$\begin{aligned} f : X^{1:I} &\to Y \\ (x_1, \ldots, x_i, \ldots, x_I) &\mapsto y = f(x_1, \ldots, x_i, \ldots, x_I) \end{aligned}$$

defines a new function whose inputs are restricted to the corresponding cross section,

$$\begin{aligned} f_{\tilde{x}_{\mathsf{i}}} : X^{\mathsf{i}^c} \times \tilde{x}_{\mathsf{i}} &\to Y \\ (x_1, \ldots, x_{i_1-1}, x_{i_1+1}, \ldots, &\mapsto y = f(x_1, \ldots, x_{i_1-1}, \tilde{x}_{i_1}, x_{i_1+1}, \ldots, \\ x_{i_J-1}, x_{i_J+1}, \ldots, x_I \quad) &\qquad x_{i_J-1}, \tilde{x}_{i_J}, x_{i_J+1}, \ldots, x_I \quad). \end{aligned}$$

If we identify the cross sections with corresponding thinned product space,

$$X^{\mathrm{i}^c} \times \tilde{x}_{\mathrm{i}} \cong X^{\mathrm{i}^c},$$

then this procedure defines functions

$$f_{\tilde{x}_{\mathrm{i}}} : X^{\mathrm{i}^c} \to Y.$$

This procedure is generally known as **partial evaluation**, although in computer science it is often referred to as **Currying**. Besides the multi-index notation I use here,

$$f_{\tilde{x}_{\mathrm{i}}},$$

it is not uncommon to see notation like

$$f(\cdot \mid \tilde{x}_{i_1}, \ldots, \tilde{x}_{i_J}),$$

where $\cdot$ represents a remaining, unbound input variable. One might also encounter notation that explicitly writes out all of the bound and unbound variables,

$$f(x_1, \ldots, x_{i_1 - 1}, \tilde{x}_{i_1}, x_{i_1 + 1}, \ldots, x_{i_J - 1}, \tilde{x}_{i_J}, x_{i_J + 1}, \ldots, x_I).$$

While this latter notation is most direct, it quickly becomes ungainly when working with more than a few components.

Partial evaluation of functions is often taken for granted in practical applications. Explicitly acknowledging it, however, can help us better understand less obvious constructions.

For instance, consider a space X equipped with a binary addition operator

$$\begin{aligned} + : X \times X &\to X \\ x_1, x_2 &\mapsto x_1 + x_2. \end{aligned}$$

Partially evaluating this function on the first input component effectively defines a function

$$\begin{aligned} t_{\tilde{x}_1} : X &\to X \\ x_2 &\mapsto \tilde{x}_1 + x_2. \end{aligned}$$

This new function *translates* each point in X by $\tilde{x}_1$. Unsurprisingly, $t_{\tilde{x}_1}$ is referred to as a translation operator.

Similarly, if we have a space X equipped with a binary multiplication operator

$$\begin{aligned} \cdot : X \times X &\to X \\ x_1, x_2 &\mapsto x_1 \cdot x_2, \end{aligned}$$

then partial evaluation effectively defines a function

$$\begin{aligned} s_{\tilde{x}_1} : X &\to X \\ x_2 &\mapsto \tilde{x}_1 \cdot x_2. \end{aligned}$$

This new function *scales* each point in X by $\tilde{x}_1$. Fittingly, $s_{\tilde{x}_1}$ is often referred to as a scaling operator.

In many applications, translations and scalings appear to be intuitive. Partial evaluation allows us to formalize that intuition, which can then helps us recognize its limitations and consequences. For example, translation isn't well-defined on a space that isn't equipped with the right algebraic structure!

7 Prototypical Product Spaces

Most of the spaces that we encounter in practical applications are product spaces built up from the prototypical spaces that we reviewed in Chapter 2, Section 2. In particular, these product spaces combine finite sets, integers, and real lines together. That said, there are a few product spaces worth particular note.

7.1 Power Spaces

Often we are interested not in any single point of a space X, but rather multiple points at the same time. The selection of I distinct points from the same space can be modeled by replicating the space I times and then using those replications as components of a product space,

$$X^I = X \times ... \times X = \prod_{i=1}^{I} X.$$

This construction is often referred to as a **replicated product space**, **identical product space**, or **power space**.

We have already encountered this construction in a few places. For example, we defined binary algebraic operator over the space X as mapping two points in X into a single point in X. That input space of pairs of points was denoted $X \times X$, which is just a binary power space with a separate copy of X for each input.

Similarly, sequences of I points from a given space,

$$\{x_1, ..., x_i, ..., x_I\},$$

can be defined as points of the power space X^I. Allowing I to approach countable infinity allows for arbitrarily long sequences.

One potential difficulty that can arise when working with power spaces is distinguishing between the different copies of the base space X that form the components. In particular, any notation that might differentiate between the individual copies a single space, such as ticks or integer indices, can also be confused as defining different spaces entirely. Depending on the context, for instance, $X_1 \times X_2$ might be used to denote both a product space comprised of two *different* component spaces or power space comprised of two copies of the *same* base space.

In practice, the most robust approach is typically to label each component space with an integer index and then explicitly communicate which, if any, component spaces are identical.

7.2 Multivariate Real Numbers

Combining I real lines together defines the **multivariate real numbers**,

$$\mathbb{R}^I = \prod_{i=1}^{I} \mathbb{R}.$$

This is also referred to as a **real space** or an **I-dimensional Euclidean space**.

The multivariate real numbers inherit the identity crisis of the real lines from which they are built. If we take the rigid real line perspective, for instance, then there will be infinitely many multivariate real numbers: different choices of component ordering, algebraic, and metric structures define distinct product spaces.

On the other hand, if we assume the flexible real line perspective, then then multivariate real numbers will correspond to a single, flexible space. In this case, each distinct configuration of the component real lines defines a different configuration of the corresponding product space. Following the real line terminology, I will refer to these as rigid real spaces and flexible real spaces, respectively.

To visually summarize the structure of a real space, we can extend metric grids defined over the component spaces across the other component spaces and then overlay them to form a rectangular *mesh* (Figure 15).

As we discussed in Section 3.1, the real space $\mathbb{R}^2$ is the one space where rectangular subsets actually correspond to Euclidean rectangles.

8 Conclusion

The construction of product spaces defines a systematic way to work with multiple spaces at the same time. Because product spaces define the sophisticated spaces we need for many practical applications, we will need to be comfortable with their structure and manipulation.

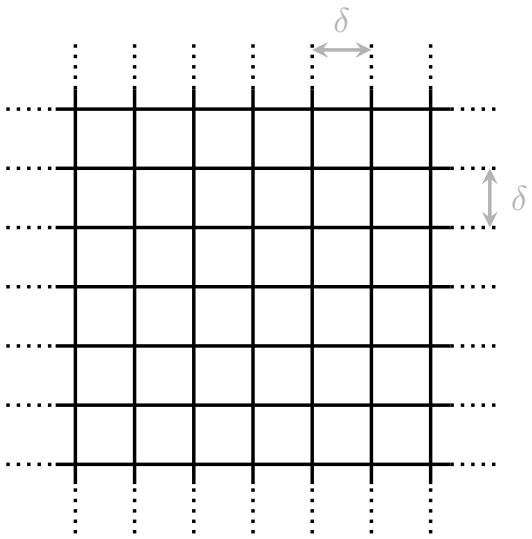


Figure 15: Extending component grids into the product space defines a rectangular mesh that visually communicate the product metric structure. Here $\mathbb{R}^2$ is represented by a two-dimensional mesh.

Working with more than a few component spaces does require some careful organization. In some cases, we can explicitly enumerate each component set with distinct variable names. In others, we can fall back on more flexible tools like multi-index notation.

Acknowledgements

I thank Alexander Noll, Pietro Monticone, and and Léo Burgund for helpful comments.

A very special thanks to everyone supporting me on Patreon: Adam Fleischhacker, Adriano Yoshino, Alan Chang, Alessandro Varacca, Alexander Bartik, Alexander Noll, Alexander Petrov, Alexander Rosteck, Anders Valind, Andrea Serafino, Andrew Mascioli, Andrew Rouillard, Andrew Vigotsky, Angie_Hyunji Moon, Ara Winter, Austin Rochford, Austin Rochford, Avraham Adler, Ben Matthews, Ben Swallow, Benjamin Glemain, Bradley Kolb, Brynjolfur Gauti Jónsson, Cameron Smith, Canaan Breiss, Cat Shark, Charles Naylor, Chase Dwelle, Chris Zawora, Christopher Mehrvarzi, Chuck Carlson, Colin Carroll, Colin McAuliffe, Cruz, Damien Mannion, Damon Bayer, dan mackinlay, Dan Muck, Dan W Joyce, Dan Waxman, Dan Weitzenfeld, Daniel Edward Marthaler, Daniel Rowe, Darshan Pandit, Darthmaluus , David Burdelski, David Galley, David Humeau, David Wurtz, dilsher singh dhillon, Doug Rivers, Dr. Jobo, Dr. Omri Har Shemesh, Ed Cashin, Ed Henry, Edgar Merkle, edith darin, Eric LaMotte, Erik Banek, Ero Carrera, Eugene O'Friel, Felipe González, Fergus Chadwick, Finn

Lindgren, Florian Wellmann, Francesco Corona, Geoff Rollins, Greg Sutcliffe, Guido Biele, Hamed Bastan-Hagh, Haonan Zhu, Hector Munoz, Henri Wallen, hs, Hugo Botha, Håkan Johansson, Ian Costley, Ian Koller, idontgetoutmuch, Ignacio Vera, Ilaria Prosdocimi, Isaac Vock, J, J Michael Burgess, Jair Andrade, James Hodgson, James McInerney, James Wade, Janek Berger, Jason Martin, Jason Pekos, Jason Wong, Jeff Burnett, Jeff Dotson, Jeff Helzner, Jeffrey Erlich, Jesse Wolfhagen, Jessica Graves, Joe Wagner, John Flournoy, Jonathan H. Morgan, Jonathon Vallejo, Joran Jongerling, Joseph Despres, Josh Weinstock, Joshua Duncan, Joshua Griffith, Josué Mendoza, JU, Justin Bois, Karim Naguib, Karim Osman, Kejia Shi, Kevin Foley, Kristian Gårdhus Wichmann, Kádár András, lizzie , LOU ODETTE, Marc Dotson, Marcel Lüthi, Marek Kwiatkowski, Mark Donoghoe, Mark Worrall, Markus P., Martin Modrák, Matt Moores, Matthew, Matthew Kay, Matthieu LEROY, Maurits van der Meer, Merlin Noel Heidemanns, Michael DeWitt, Michael Dillon, Michael Lerner, Mick Cooney, Márton Vaitkus, N Sanders, Name, Nathaniel Burbank, Nic Fishman, Nicholas Clark, Nicholas Cowie, Nick S, Nicolas Frisby, Octavio Medina, Ole Rogeberg, Oliver Crook, Olivier Ma, Pablo León Villagrá, Patrick Kelley, Patrick Boehnke, Pau Pereira Batlle, Peter Smits, Pieter van den Berg , ptr, Putra Manggala, Ramiro Barrantes Reynolds, Ravin Kumar, Raúl Peralta Lozada, Riccardo Fusaroli, Richard Nerland, RLW, Robert Frost, Robert Goldman, Robert kohn, Robert Mitchell V, Robin Taylor, Ross McCullough, Ryan Grossman, Rémi , S Hong, Scott Block, Scott Brown, Sean Pinkney, Sean Wilson, Seth Axen, shira, Simon Duane, Simon Lilburn, Srivatsa Srinath, sssz, Stan_user, Stefan, Stephanie Fitzgerald, Stephen Lienhard, Steve Bertolani, Stone Chen, Susan Holmes, Svilup, Sören Berg, Tao Ye, Tate Tunstall, Tatsuo Okubo, Teresa Ortiz, Thomas Lees, Thomas Vladeck, Tiago Cabaço, Tim Radtke, Tobychev , Tom McEwen, Tony Wuersch, Utku Turk, Virginia Fisher, Vitaly Druker, Vladimir Markov, Wil Yegelwel, Will Farr, Will Tudor-Evans, woejozney, Xianda Sun, yolhaj , yureq , Zach A, Zad Rafi, and Zhengchen Cai.

License